

OS Assignment 6

2401MC35

WriteUps for Q3 and Q4 :

Question 3 — Deadlock Prevention via Resource Ordering

1. The Deadlock Condition (Coffman Conditions)

For a deadlock to occur, four conditions must hold simultaneously:

- **Mutual Exclusion:** Resources cannot be shared simultaneously.
- **Hold and Wait:** A process holds at least one resource while waiting to acquire another.
- **No Preemption:** Resources cannot be forcibly taken from a process holding them.
- **Circular Wait:** A closed chain of processes exists such that each process holds at least one resource that is needed by the next process in the sequence ($P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_n \rightarrow P_0$).

In **resourceorder_bad.c**, Process A acquires Lock 1 and requests Lock 2, while Process B acquires Lock 2 and requests Lock 1. By delaying execution between acquisitions, a circular dependency is formed: Process A waits on Process B, and Process B waits on Process A. The system permanently hangs.

2. Why Resource Ordering Eliminates Circular Wait

Deadlock prevention works by invalidating at least one of the four Coffman conditions. Resource ordering targets the Circular Wait condition:

- Let $R = \{R_1, R_2, \dots, R_m\}$ be the set of all resource types in the system.
- We define a strict global total ordering function $F: R \rightarrow \mathbb{N}$ such that every resource is mapped to a unique integer (e.g., $F(\text{Lock1}) = 1$, $F(\text{Lock2}) = 2$).
- The protocol mandates that any process requiring multiple resources must request them in strictly increasing order of $F(R)$. If a process holds resource R_i , it may only request resource R_j if $F(R_j) > F(R_i)$.

Proof by Contradiction:

Assume a circular wait exists under this protocol:

$P_0 \text{ holds } R_0 \text{ and waits for } R_1 \implies F(R_0) < F(R_1)$

$P_1 \text{ holds } R_1 \text{ and waits for } R_2 \implies F(R_1) < F(R_2)$

$\dots$

$P_k \text{ holds } R_k \text{ and waits for } R_0 \implies F(R_k) < F(R_0)$

By transitivity along the chain:

$F(R_0) < F(R_1) < F(R_2) < \dots < F(R_k) < F(R_0)$

This implies $F(R_0) < F(R_0)$, which is impossible. Therefore, a cycle cannot form in the allocation graph, and deadlock is mathematically eliminated as demonstrated in [resourceorder_fixed.c](#).

Question 4 — Synchronization Strategy and Proof of Deadlock Freedom

1. System Architecture & Resource Pool The system consists of 5 processes competing across multiple iterations for 3 shared resource pools:

- Resource 0 (Printer): 2 instances
- Resource 1 (Scanner): 1 instance
- Resource 2 (Disk): 2 instances

Each process requires a combination of 2 resources to perform its work cycle. Mutual exclusion over pool counters is guaranteed using an atomic test-and-set (`xchg`) spinlock (`table_lock`).

2. Deadlock Prevention Strategy: Strict Hierarchical Resource Ordering We implemented Hierarchical Resource Ordering across all processes:

- Every resource type is assigned a strict index:
$$\text{Printer} = 0, \quad \text{Scanner} = 1, \quad \text{Disk} = 2$$
- When a process needs two resources (R_A, R_B) , the acquisition helper evaluates:
$$\text{First} = \min(R_A, R_B), \quad \text{Second} = \max(R_A, R_B)$$
- Regardless of how the requirements are declared in application logic, every process requests the lower-indexed resource instance first, followed by the higher-indexed resource instance.

3. Justification & Proof of Correctness

- **Elimination of Circular Wait:** Because every process requests instances in the order $R_0 \rightarrow R_1 \rightarrow R_2$, no process holding an instance of R_2 can ever wait for an instance of R_1 or R_0 . Thus, a directed dependency graph among the processes is strictly a Directed Acyclic Graph (DAG) with respect to resource types.
- **Absence of Starvation:** Processes yield the CPU via `sleep(1)` when instances are currently exhausted, allowing xv6's round-robin scheduler to switch context to the running processes that currently hold those instances. Once their work cycle completes (simulated with `sleep(3)`), all resources are released via `release_resources()`, waking waiting processes.
- **Integrity of Resource Counters:** Critical sections incrementing and decrementing available pool instances are serialized using the `table_lock` spinlock, preventing race conditions and ensuring allocated counts never exceed the physical limits (2, 1, 2).

